

This homework contains a theory part (Q1-Q4) and a programming part (Q5). Both parts must be uploaded to Canvas before the deadline. For the programming portion, please use the starter code template provided on Canvas. You should not change the file names and function signatures. you should not use packages for encapsulation. Please zip and name the source code as [EID1 EID2].zip and submit with a PDF containing the answers to the theory questions.

1. (a) Prove or disprove that every symmetric and transitive relation is reflexive.

Reflexivity = every element has a relation to itself

Transitivity = if $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ must exist (shortcuts exist)

Symmetric = if $A \rightarrow B$, then $B \rightarrow A$ must exist

A relation where all elements have a symmetric 2-way street AND transitive property would mean $A \rightarrow B$ and $B \rightarrow A$ would also have an $A \rightarrow A$ and $B \rightarrow B$ in vice versa. However, the loophole exists where an element with no streets connected to it still fulfills the symmetric and transitive criteria but does not enforce reflexivity. FALSE.

(b) Prove or disprove that every irreflexive and transitive relation is asymmetric.

Irreflexivity = no self-relations

Transitivity = if $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ must exist (shortcuts exist)

Asymmetric = if $A \rightarrow B$, then $B \rightarrow A$ must NOT exist, AND no self-relations

For there to be an irreflexive and transitive relation, a 2-way street is impossible because $A \rightarrow B$ and $B \rightarrow A$ would require $A \rightarrow A$ to exist. Because it is irreflexive, all cases of 2-way streets are eliminated, thus fulfilling the Asymmetric criteria. TRUE.

2. Suppose that the underlying communication system guarantees FIFO ordering of messages. How will you exploit this feature to reduce the communication complexity of the vector clock algorithm? Give an expression for overhead savings if your scheme is used instead of the traditional vector clock algorithm. Assume that any process can send at most m messages. (Hint: You should use only $O(n)$ storage at each process.)

Normally, each process maintains its own vector clock v individually and updates it when a clock vector message is received. This requires a vector of n int values per message. We can leverage the FIFO communication system in this scenario by utilizing the sequential ordering of messages. This way, process P_i only needs to send its own vector clock component instead of the whole vector.

Vector Clock Alg. FIFO communication

P_i ::

var

n : int; // n represents # of processes

v : array[1.. n] of int, initially 0; // v represents P_i 's vector clock

initially

$v[i] = 1$ at P_i , and $v[j] = 0$ for every other j .

1 Upon receive APP(δ) from P_j

 // δ is P_j 's vector-clock value

 // FIFO guarantees that earlier messages from P_j have already arrived

```

2   v [j] := max( v[j] , delta );
3   v [i] := v [i] + 1;
4 procedure sendEvent(i, j)
    // Send Pi's vector clock component 'v [i]' instead of entire v
5   send APP(v [i]) to Pj ;
6   v [i] := v [i] + 1; // update internal clock component
7 procedure internalEvent(i)
8   v [i] := v [i] + 1 // update internal clock component

```

Original vector clock algorithm had a overhead complexity of $O(nm * n) = O(mn^2)$. Because FIFO ordering allows us to send 1 integer instead of an array of n integers our augmented overhead expression would be $O(nm * 1) = O(nm)$.

3. Some applications require two types of accesses to the critical section – read access and write access. For these applications, it is reasonable for multiple read accesses to happen concurrently. However, a write access cannot happen concurrently with either a read access or a write access. Modify Lamport's mutex algorithm for such applications.

We add request types to distinguish READs from WRITES. A WRITE request can only access the critical section if its first in queue. Where READ requests can gain access concurrently to the critical section so long as there is NO WRITE REQUESTS ahead of them in queue. This prevents the possibility of data race conditions.

Lamport's algorithm with read/write mutex

Pi ::

var

 n: int;

 q: list of Req;

 numAcks: int, initially 0;

1 Upon receive REQUEST (ts, type) from Pj

2 q.insert(new Req(ts, j, type));

3 send ACK (logicalClock()) to Pj ;

4 Upon receive ACK (ts) from Pj

5 numAcks := numAcks + 1;

6 if (numAcks = n - 1) $\wedge$ allowAccess(i) then enterCriticalSection();

7 Upon receive RELEASE from Pj

8 if $\exists r \in q : r.pid = j$ then q.remove(r);

9 if (numAcks = n - 1) $\wedge$ allowAccess(i) then enterCriticalSection();

10 function allowAccess(i) // instead of calling first in queue we check a request type

11 myRequest = request in q where myRequest.pid = i;

12 if myRequest.type = WRITE then

13 // a WRITE request can only enter if its first in queue

14 Return q.first().pid := i;

15 if myRequest.type := READ then

16 // a READ request can enter only if no WRITE ahead in queue

```

17      return  $\neg \exists$  earlierRequest  $\in$  q : earlierRequest.type = WRITE  $\wedge$  earlierRequest <
myRequest;
18 procedure requestCS(i)
19   int ts := logicalClock();
20   forall j  $\in$  [1..n] : if j  $\neq$  i then send REQUEST (ts, type) to Pj ;
21   q.insert(new Req(ts, i, type));
22   numAcks := 0;
23 procedure releaseCS(i)
24   if  $\exists$  r  $\in$  q : r.pid = i then q.remove(r );
25   forall j  $\in$  [1..n] : if j  $\neq$  i then send RELEASE () to Pj ;

```

4. (a) Extend Lamport's mutex algorithm to solve k-mutual exclusion problem which allows at most k processes to be in the critical section concurrently.

Allowing the first k requests access to critical section

Lamport's Algorithm with k-mutual exclusion

Pi ::

var

n: int;

k: int;

q: list of Req;

numAcks: int, initially 0;

1 Upon receive REQUEST (ts) from Pj

2 q.insert(new Req(ts, j));

3 send ACK (logicalClock()) to Pj ;

4 Upon receive ACK (ts) from Pj

5 numAcks := numAcks + 1;

6 if (numAcks = n - 1) $\wedge$ withinFirstK(i) then enterCriticalSection();

7 Upon receive RELEASE from Pj

8 if $\exists$ r $\in$ q : r.pid = j then q.remove(r);

9 if (numAcks = n - 1) $\wedge$ withinFirstK(i) then enterCriticalSection();

10 function withinFirstK(i)

11 return i among the first k requests in q; // instead of returning the boolean of first in queue
we return the boolean of in the first k subset range of queue

12 procedure requestCS(i)

13 int ts := logicalClock();

14 forall j $\in$ [1..n] : if j $\neq$ i then send REQUEST (ts) to Pj ;

15 q.insert(new Req(ts, i));

16 numAcks := 0;

17 procedure releaseCS(i)

18 if $\exists$ r $\in$ q : r.pid = i then q.remove(r);

19 forall j $\in$ [1..n] : if j $\neq$ i then send RELEASE () to Pj

(b) Extend Ricart and Agrawala's mutex algorithm to solve the k-mutual exclusion problem.

There is no global request queue in Ricart and Agrawala's algorithm. It works by using priority/timestamps stored in *myts* and sending OKAY if *Pj* has higher priority than *Pi*, *Pj* gains access to the critical section once it receives *n-1* OKAYs, leading to a lot of communication overhead. To apply *k*-mutual exclusion means to grant permission to the *k* highest-priority requests, meaning *Pj* needs *n-k* OKAYs before entering the critical section.

Ricart and Agrawala with *k*-mutual exclusion

Pi ::

var

n: int;

k: int;

 pendingQ: list of int;

myts: int, initially ∞ ;

 numOkay : int, initially 0;

1 Upon receive REQUEST (*jts*) from *Pj*

2 if (*jts* < *myts*) $\vee$ ((*jts* = *myts*) $\wedge$ (*j* < *i*)) then send OKAY () to *Pj* ;

3 else pendingQ.add(*j*);

4 Upon receive OKAY from *Pj*

5 numOkay := numOkay + 1;

6 if numOkay := *n* - *k* then enterCriticalSection(); // adjust the *n-1* expression to *n-k*

7 procedure requestCS(*i*)

8 *myts* := logicalClock();

9 forall *j* $\in$ [1..*n*] : if *j* $\neq$ *i* then send REQUEST (*myts*) to *Pj* ;

10 numOkay := 0;

11 procedure releaseCS(*i*)

12 *myts* := ∞ ;

13 forall *j* $\in$ pendingQ : send OKAY () to *Pj* ;

14 pendingQ.clear ();